# Capstone Project: Library Management System using ASP.NET Core Web API and Angular Using Clean Architecture

## 1. Introduction

The Library Management System (LMS) streamlines the process of managing books, members, loans, and returns. The system allows administrators to manage books, categories, and users, while members can search for books, borrow them, and track their loan history.

## 2. Technology Stack

- Backend: ASP.NET Core Web API
- Frontend: Angular
- Database: SQL Server
- ORM: Entity Framework Core
- Authentication: JWT Authentication
- Exception Handling: Global exception handling middleware
- Architecture: Clean Architecture

## 3. Clean Architecture Implementation

### a) Domain Layer (Core Business Logic)

Defines entities such as Book, Member, Loan, etc.
Contains business rules and validation logic.

#### *Models:*

- Book (Id, Title, Author, ISBN, CategoryId, PublishedYear, CopiesAvailable)
- Category (Id, Name, Description)
- Member (Id, Name, Email, MembershipDate, Status)
- Loan (Id, MemberId, BookId, LoanDate, DueDate, ReturnDate, Status)

### b) Application Layer (Use Cases and Services)

Implements interfaces for business logic.
Contains services such as BookService, MemberService, LoanService.
Uses DTOs to communicate between layers.

#### *Interfaces:*

```csharp
public interface IBookService
{
    Task<IEnumerable<BookDto>> GetAllBooksAsync();
    Task<BookDto> GetBookByIdAsync(int id);
    Task AddBookAsync(BookDto book);
```

```
    Task UpdateBookAsync(BookDto book);
    Task DeleteBookAsync(int id);
}

public interface ILoanService
{
    Task<IEnumerable<LoanDto>> GetMemberLoansAsync(int memberId);
    Task<LoanDto> BorrowBookAsync(LoanDto loan);
    Task ReturnBookAsync(int loanId);
}
```

#### c) Infrastructure Layer (Database and External Services)

Implements repositories for data access using Entity Framework Core.

***Repository Interfaces:***

- IBookRepository
- IMemberRepository
- ILoanRepository

#### d) Presentation Layer (API Controllers)

Exposes endpoints for the Angular frontend.
Uses MediatR for CQRS (Command Query Responsibility Segregation).

***Controllers:***

- BooksController
- MembersController
- LoansController

## 4. Backend Implementation (ASP.NET Core Web API)

- Authentication & Authorization: Implement using JWT tokens.
- Controllers & Endpoints: RESTful API with endpoints for book management, loan processing, and member authentication.
- Dependency Injection: Ensures loose coupling.

## 5. Exception Handling in Web API

- Custom Exceptions: Defines BookNotFoundException, LoanProcessingException.

## 6. Frontend Implementation (Angular)

- Angular Modules: Organized into feature-based modules such as BookModule, MemberModule, LoanModule.
- Services: HTTP services for API communication.

### *Angular Services:*

- BookService
- MemberService
- AuthService
- LoanService

## 7. Database Design

- Tables: Books, Categories, Members, Loans.
- Relationships: One-to-many between Categories and Books, one-to-many between Members and Loans.